CAPTEURS ET MESURE · TRAVAUX PRATIQUES

01 Capturer une première mesure physique : la température

Durée · 4 h **Niveau** · ZZ2 **Travail** · en binôme

Rendu · avant la séance suivante

Ce premier TP introduit les capteurs de température, analogiques et numériques, que nous utiliserons tout au long du module. Vous mettrez en place la chaîne de développement, réaliserez vos premières mesures et construirez la chaîne de visualisation côté PC.

OBJECTIFS

- Mettre en place la chaîne de développement.
- Découvrir les capteurs de température analogiques et numériques.
- Acquérir les premières mesures.
- Construire la chaîne de visualisation côté PC.

MATÉRIEL ET ENVIRONNEMENT

- Carte WisBlock RAKwireless **RAK11310** : microcontrôleur RP2040 et module radio (non utilisé en ZZ2).
- Un capteur de température et d'humidité **SHTC3** (Sensirion).
- Un assistant IA (LLM) de votre choix : Claude, ChatGPT, Mistral…
- Un câble USB.
- Un poste Windows, Linux ou macOS : votre machine ou le PC de la salle.

À RENDRE

Un compte rendu par binôme, à déposer avant le début de la séance de TP suivante.

1 Configurer l'environnement Arduino

Cette partie d'installation se fait pas à pas et ne comporte aucune question. Elle suit le Quick Start Guide du RAK11310 et le guide d'installation du BSP. Votre objectif : programmer la carte avec succès.

Étape 1 — Installer l'IDE Arduino

Téléchargez l'IDE Arduino 2.x depuis le site officiel et installez-le.

- **Windows** : lancez l'installeur (I Agree › Next › Install).
- **Linux** : il suffit d'extraire l'archive. Sur certaines distributions, installez d'abord pySerial : `pip3 install pyserial`.
- **macOS** : décompressez l'archive et déplacez `Arduino.app` dans le dossier Applications.

> **ATTENTION — WINDOWS**
>
> N'installez **pas** l'IDE depuis le Microsoft Store : cette version gère mal les paquets de cartes tiers. Utilisez l'installeur du site officiel.

Étape 2 — Ajouter le paquet RAKwireless

Ouvrez **Fichier › Préférences**. Cliquez sur l'icône à côté du champ *URL de gestionnaire de cartes supplémentaires* et collez l'URL ci-dessous. S'il y a déjà d'autres URL, ajoutez celle-ci sur une nouvelle ligne, puis validez avec **OK**.

URL DU GESTIONNAIRE DE CARTES — JSON

```
https://raw.githubusercontent.com/RAKwireless/RAKwireless-Arduino-BSP-Index/main/package_rakwireless_index.json
```

Code 1 — Index des paquets RAKwireless.

Dans la même fenêtre, cochez l'affichage détaillé (*verbose*) pour la **compilation** et le **téléversement**. Vous verrez ainsi les avertissements et les erreurs, et ce qui se passe à chaque étape.

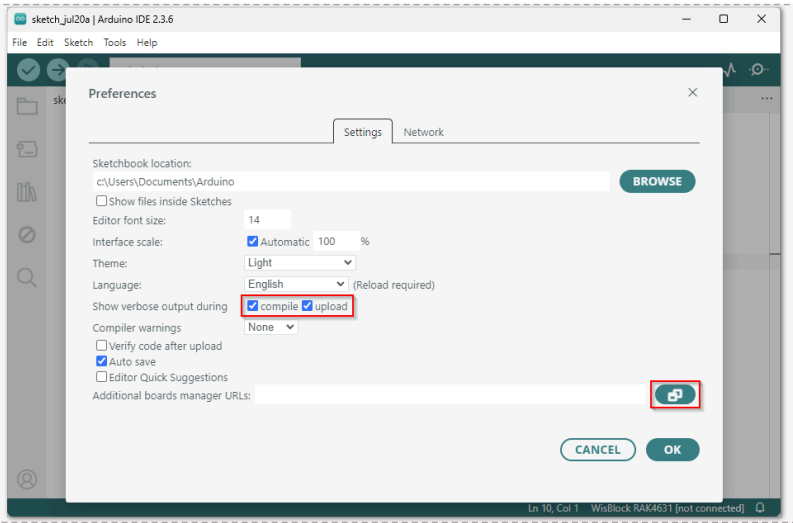


Figure 1 — Préférences : sorties détaillées et accès aux URL du gestionnaire de cartes. Source : RAKwireless.

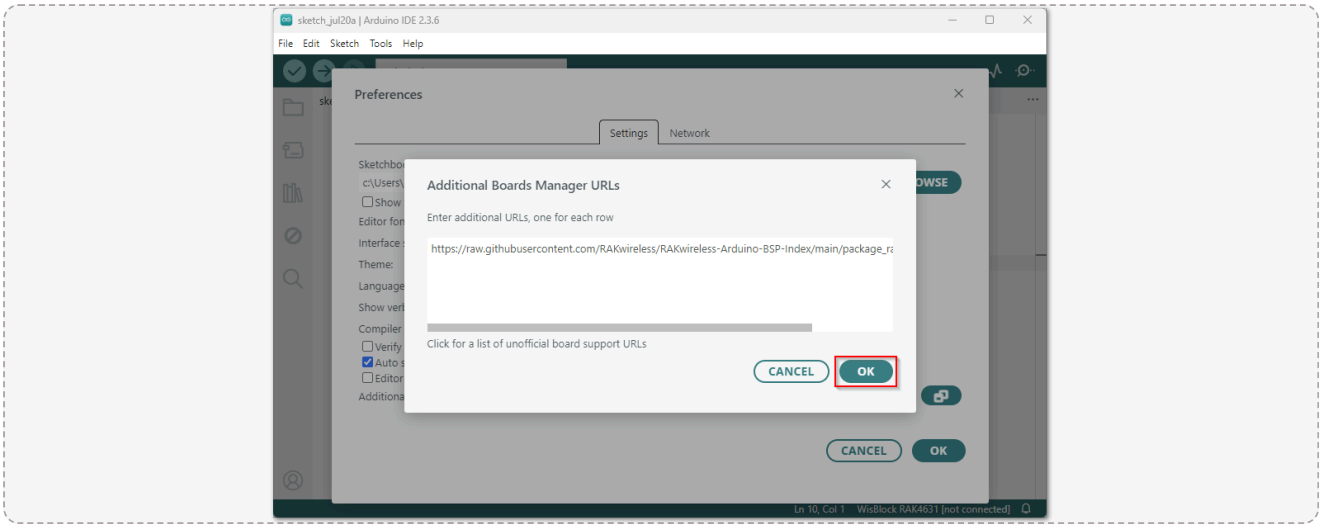


Figure 2 — Ajout de l'URL du paquet RAKwireless. Source : RAKwireless.

Étape 3 — Installer le BSP

Redémarrez l'IDE. Cliquez sur l'icône **Gestionnaire de cartes** dans la barre latérale, tapez **RAK11300** dans la recherche, puis installez la dernière version de **RAKwireless RP2040 Boards**. L'installation prend environ 5 minutes.



Figure 3 — Installation du BSP RAKwireless RP2040 Boards depuis le gestionnaire de cartes. Source : disk91.com.

Sélectionnez ensuite la carte dans **Outils › Carte › RAKwireless RP2040 Modules › WisBlock RAK11300**. Le RAK11310 est basé sur le module RAK11300 et partage le même firmware. L'IDE utilise alors le bon compilateur et les bons réglages.

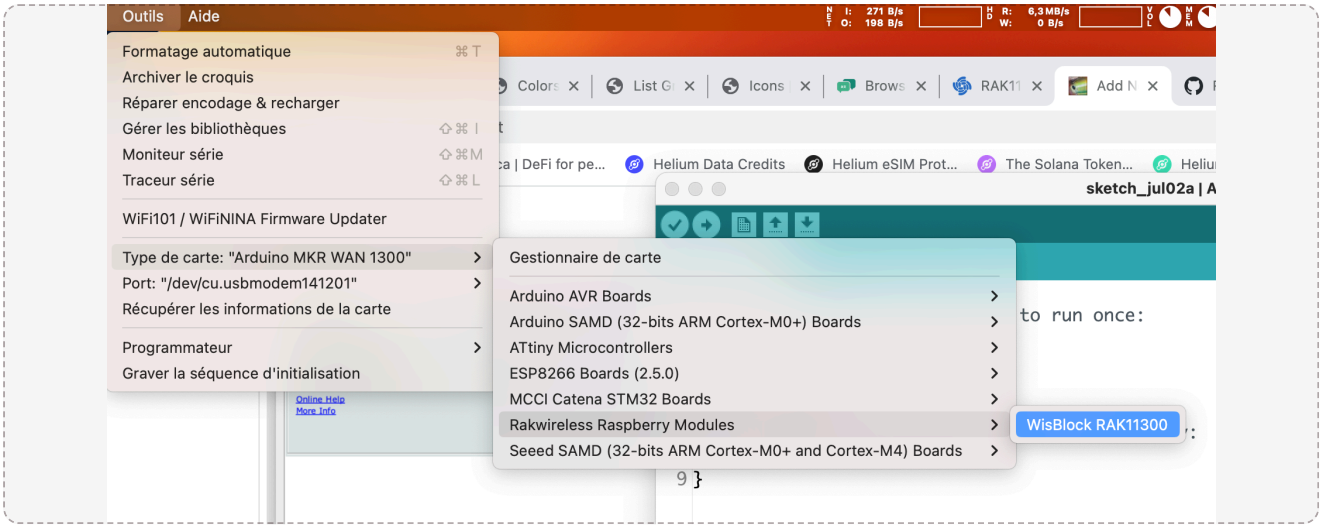


Figure 4 — Sélection de la carte WisBlock RAK11300. Source : disk91.com.

ATTENTION — PILOTE USB SOUS WINDOWS

Après l'installation du BSP, exécutez `post_install.bat` **en tant qu'administrateur**. Sinon, le téléversement risque d'échouer. Emplacement (remplacez *username* et adaptez le numéro de version) :

```
C:\Users\username\AppData\Local\Arduino15\packages\rakwireless\hardware\mbed_rp2040\0.0.6\post_install.bat
```

Vérifier l'installation

Avant d'ouvrir un exemple, vérifiez dans **Outils › Carte** que la carte **WisBlock RAK11300** est bien sélectionnée. Les exemples proposés dans **Fichier › Exemples** dépendent de la carte choisie : avec une autre carte, les exemples RAKwireless n'apparaissent pas et la compilation cible le mauvais microcontrôleur.

1. Ouvrez l'exemple qui fait clignoter la LED : **Fichier › Exemples › Basics › Blink**.
2. Branchez la carte en USB et sélectionnez son port dans **Outils › Port**.
3. Cliquez sur **Téléverser** (flèche) : l'IDE compile puis programme la carte.
4. Vérifiez que la LED clignote. La carte de base comporte deux LED utilisables : `LED_GREEN` et `LED_BLUE`.

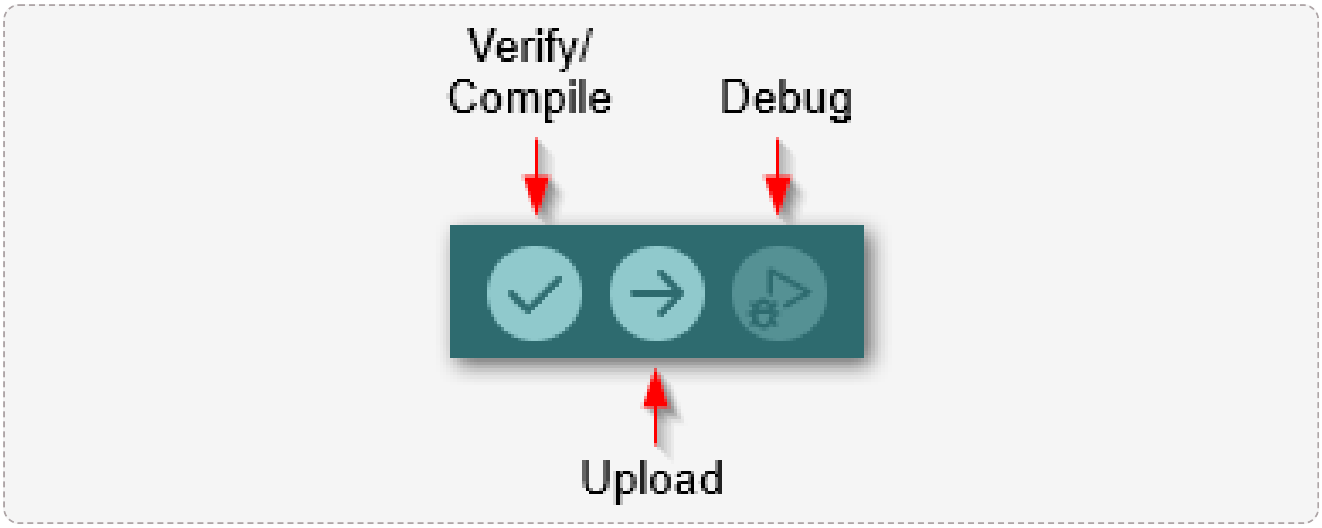


Figure 5 — Boutons de l'IDE : Vérifier (compiler), Téléverser, Déboguer. Source : RAKwireless.

ASTUCE — EN CAS D'ÉCHEC DU TÉLÉVERSEMENT

- Vérifiez d'abord que le bon port est sélectionné dans **Outils › Port**.
- Si besoin, appuyez **deux fois** sur le bouton **Reset** de la carte de base. Le module passe en mode bootloader : un lecteur USB `RPI-RP2` apparaît et la carte redevient programmable.

DÉBOGAGE

Le débogage passe par le port série : le RP2040 utilise son USB natif (`Serial`) pour la programmation et pour les traces. Observez les données échangées avec le **Moniteur série** de l'IDE (**Outils › Moniteur série**).

2 Lire la température interne du RP2040

Le RP2040 intègre un capteur de température : une diode dont la tension V_{BE} varie avec la température de la puce. Cette tension est analogique. Elle est reliée au **canal 4** de l'ADC interne, que nous allons lire pour obtenir une première mesure.

Cette partie s'appuie sur le tutoriel de H.-P. Halvorsen, transposé en C pour l'environnement Arduino. La formule de conversion est donnée dans la datasheet du RP2040 (§ 4.9.5).

De la tension à la température

La tension vaut typiquement **0,706 V à 27 °C** et diminue de **1,721 mV par degré**. La relation est linéaire, on l'inverse donc directement :

```
T (°C) = 27 - (V - 0,706) / 0,001721
```

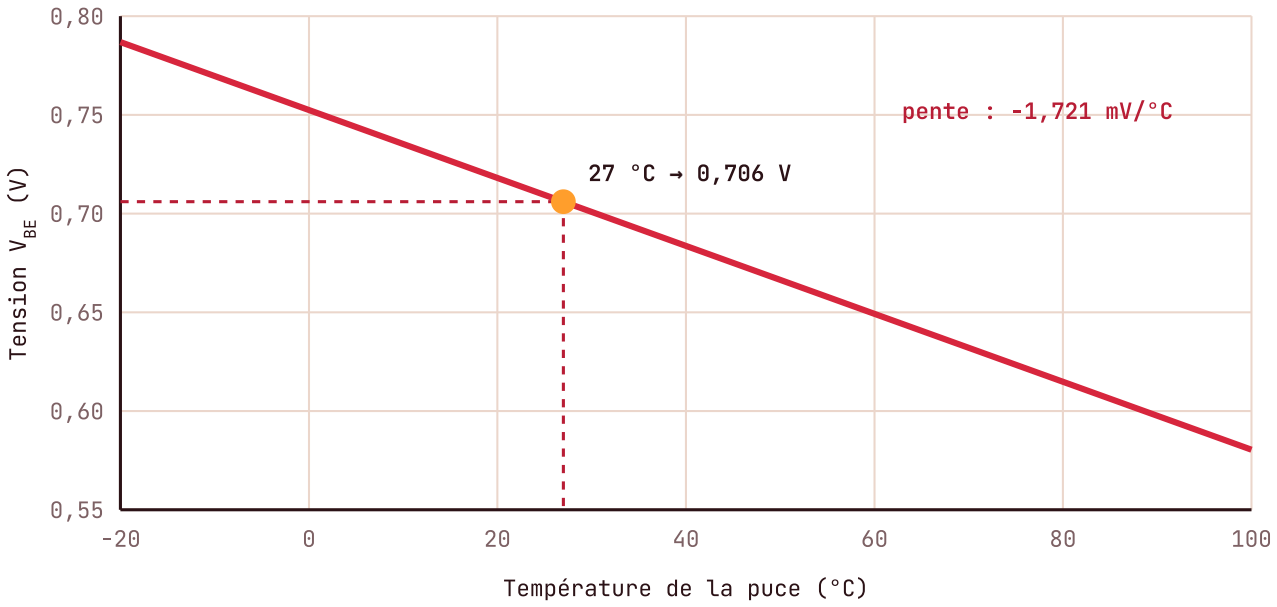


Figure 6 — Tension du capteur interne en fonction de la température (valeurs typiques de la datasheet).

De la valeur brute à la tension

L'ADC du RP2040 a une résolution de **12 bits** : il renvoie une valeur entière entre 0 et 4095 pour une tension entre 0 et 3,3 V. On obtient la tension par V = brut × 3,3 / 4096.

> **ASTUCE — RÉSOLUTION DE L'ADC**
>
> `adc_read()` renvoie la valeur sur 12 bits. Divisez par 4096 (2^{12}), pas par 1024 comme sur un Arduino Uno 10 bits.

Programme Arduino

Le BSP RAKwireless repose sur le SDK Pico : on peut appeler directement les fonctions ADC du RP2040. Créez un nouveau croquis, copiez le code ci-dessous, téléversez-le, puis ouvrez le **Moniteur série** à **115 200 bauds**.

TEMPERATURE_INTERNE.INO C / ARDUINO

```
#include "hardware/adc.h"   // API ADC du SDK Pico, incluse dans le BSP

const float VREF = 3.3f;           // tension de référence de l'ADC
const float LSB  = VREF / 4096.0f; // ADC 12 bits : 0 à 4095

void setup() {
  Serial.begin(115200);
  while (!Serial) {}               // attendre l'ouverture du moniteur série
  adc_init();
  adc_set_temp_sensor_enabled(true); // alimenter le capteur interne
  adc_select_input(4);               // canal 4 = capteur de température
}

void loop() {
  uint16_t brut = adc_read();              // valeur brute 0..4095
  float v = brut * LSB;                    // tension en volts
  float t = 27.0f - (v - 0.706f) / 0.001721f; // formule de la datasheet
  Serial.print("ADC=");  Serial.print(brut);
  Serial.print("  V=");  Serial.print(v, 4);
  Serial.print("  T=");  Serial.println(t, 1);
  delay(1000);
}
```

Code 2 — Lecture du capteur interne et affichage sur le port série.

ATTENTION

Le capteur mesure la température **de la puce**, pas celle de la pièce. La valeur est généralement supérieure de quelques degrés et dépend de l'activité du processeur. Elle est aussi sensible aux variations de la tension de référence : une erreur de 1 % sur 3,3 V décale la mesure d'environ 4 °C.

Q1. Relevez une dizaine de mesures dans le Moniteur série. Posez un doigt sur le module, puis retirez-le : décrivez l'évolution de la température.

Q2. Quelle valeur brute l'ADC renvoie-t-il à 27 °C ?

Q3. Calculez la résolution de la mesure : de combien de degrés la température varie-t-elle pour un pas de l'ADC ? Que pensez-vous de la stabilité des valeurs affichées ?

3 Transmettre les mesures vers le PC

Jusqu'ici, la carte affiche du texte destiné à un humain. Nous voulons maintenant qu'un **programme** sur le PC reçoive les mesures par le port série et les affiche sous forme de **graphique**, en temps réel.

Vous choisissez librement le langage et les bibliothèques côté PC. L'application sera développée entièrement avec un assistant IA (*vibe coding*). Seule contrainte : les données passent par le port série. Pour obtenir rapidement un résultat qui fonctionne, il faut d'abord définir un **protocole de communication** : une structure des données que la carte et le PC comprennent de la même façon.

Ce que transporte une liaison série

Une liaison série transporte un **flux d'octets**, rien de plus. Elle n'a aucune notion de nombre, de type ou de message. `Serial.print(23.4)` n'envoie pas un nombre : il envoie les caractères ASCII `'2' '3' '.' '4'`. `Serial.write()` envoie des octets bruts. Dans les deux cas, le récepteur doit savoir **où commence et où finit** chaque mesure, et **comment interpréter** les octets reçus.

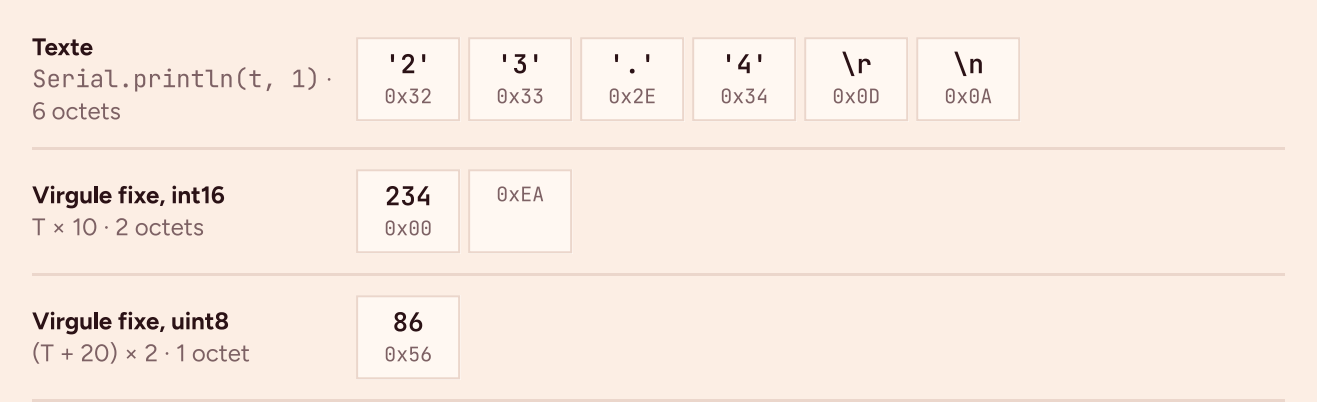


Figure 7 — La même mesure, 23,4 °C, sous trois représentations. Le récepteur ne voit que les octets.

Pourquoi éviter float et double sur un microcontrôleur

Le cœur Cortex-M0+ du RP2040 n'a **pas d'unité de calcul flottant** (FPU). Chaque opération sur un `float` est émulée par une routine logicielle : elle prend des dizaines de cycles, consomme plus d'énergie et alourdit le programme. Un `double` coûte encore plus cher.

Cette précision ne sert à rien ici. Notre mesure a une résolution d'environ 0,5 °C et une exactitude de quelques degrés (partie 2). Un `float` offre environ 7 chiffres significatifs, dont 5 décrivent du bruit. On préfère donc travailler en **entiers**.

Type C	Taille	Calcul sur RP2040	Commentaire
`double`	`8 octets`	`logiciel, très lent`	~15 chiffres significatifs, inutiles ici
`float`	`4 octets`	`logiciel, lent`	~7 chiffres significatifs, toujours inutiles
`int16_t` (T × 10)	`2 octets`	`natif, 1 cycle`	0,1 °C de pas, de −3276,8 à 3276,7 °C
`uint8_t` (à définir)	`1 octet`	`natif, 1 cycle`	plage et pas à choisir avec soin

Tableau 1 — Coût des représentations possibles d'une température.

La virgule fixe

Un entier peut porter une partie décimale si l'on convient d'une échelle : c'est la **virgule fixe**. En stockant T × 10, la valeur 234 signifie 23,4 °C. Le PC n'a qu'à diviser par 10 pour l'affichage. Avec un décalage et une échelle bien choisis, un seul octet peut même suffire (figure 7). L'idéal est de calculer cette valeur entière directement à partir de la valeur brute de l'ADC, sans jamais passer par un `float`.

Valeurs sur plusieurs octets : la notion de trame

Quand une valeur occupe **plusieurs octets**, l'émetteur et le récepteur doivent rester alignés. Si le PC ouvre le port au milieu d'une valeur, ou perd un octet, il se décale d'un octet : toutes les valeurs suivantes sont mal décodées.



Figure 8 — Un décalage d'un seul octet suffit pour que toutes les valeurs sur 2 octets soient fausses.

Pour transmettre des valeurs de plusieurs octets, on les regroupe donc en **trames**, repérées par un marqueur de début ou un séparateur. Avec des valeurs sur **un seul octet**, chaque octet reçu est une mesure complète et le décalage est impossible. C'est le cas visé dans ce TP : le problème ne vous concerne pas ici, mais gardez-le en tête pour les prochains TP.

TRAVAILLER AVEC UN LLM

Servez-vous d'un LLM pour poser des questions, explorer des pistes et remettre vos choix en question, pas pour obtenir directement la solution finale. Construisez votre protocole par itérations.

- *« Comment représenter une température sur un seul octet avec un pas de 0,5 °C ? »*
- *« Quelles bibliothèques permettent de lire un port série et de tracer une courbe en temps réel ? »*
- *« Mon programme PC reçoit des valeurs incohérentes : comment vérifier les octets reçus ? »*

ATTENTION

Le Moniteur série de l'IDE et votre application ne peuvent pas ouvrir le port en même temps. Fermez le moniteur avant de lancer votre programme PC.

Q4. Combien d'octets `Serial.println(t, 1)` envoie-t-il pour 23,4 °C ? Comparez avec une représentation en virgule fixe sur 2 octets.

Q5. Choisissez une représentation en virgule fixe adaptée à la mesure : échelle, décalage éventuel, plage couverte, nombre d'octets. Justifiez vos choix au regard de la résolution mesurée en Q3.

Q6. Décrivez votre protocole : contenu de chaque octet, ordre d'envoi, cadence. Donnez un exemple de flux en hexadécimal et la valeur décodée correspondante.

Q7. Réalisez l'application PC qui affiche la température en temps réel sous forme de courbe. Joignez une capture d'écran.

Q8. Résumez votre démarche avec le LLM : questions posées, pistes envisagées puis écartées, et pourquoi.

4 Exploiter la datasheet du SHTC3

Le **SHTC3** de Sensirion est un capteur numérique de température et d'humidité. Il fait la mesure et la conversion lui-même, puis transmet le résultat sur un bus I²C. Avant de le programmer, il faut savoir lire sa datasheet, qui vous est fournie.

Lisez-la attentivement, puis répondez aux questions suivantes. Pour chaque valeur, indiquez la page et le tableau ou la figure d'où elle provient.

HYPOTHÈSES DE CALCUL

- Valeurs **typiques** de la datasheet, à 25 °C sous 3,3 V.
- Pendant un réveil, le capteur consomme son courant de mesure en mode normal.
- Batterie idéale de **100 mAh** : capacité entièrement utilisable, sans autodécharge.

Q9. Quelle est la précision de la mesure de température à **45 °C** ? Donnez la valeur typique et la valeur maximale.

Q10. Quel est le courant consommé par le capteur lorsqu'il reste en permanence en mode **idle**, sans effectuer de mesure ?

Q11. Dans ce cas, combien de temps faut-il pour décharger complètement une batterie de 100 mAh ? Détaillez le calcul.

RÉSULTAT ATTENDU environ 2 200 h, soit à peu près 3 mois.

Q12. Second scénario : le capteur reste en mode **sleep** et ne se réveille qu'**une fois par heure, pendant 20 ms**. Calculez le courant moyen, la consommation d'énergie moyenne, puis l'autonomie avec la même batterie. Détaillez chaque étape.

RÉSULTAT ATTENDU courant moyen ≈ 0,30 µA (≈ 1 µW sous 3,3 V), autonomie ≈ 330 000 h, soit environ 38 ans.

Q13. Que se passe-t-il si l'on effectue **une mesure par minute** au lieu d'une mesure par heure ? Recalculez le courant moyen et l'autonomie, et évaluez l'impact sur la durée de vie de la batterie.

`RÉSULTAT ATTENDU` courant moyen ≈ 0,44 µA (≈ 1,5 µW), autonomie ≈ 226 000 h, soit environ 26 ans.

Q14. Ces chiffres sont-ils réalistes pour un capteur réel ? Quel est le principal poste de consommation que nous n'avons pas pris en compte, alors qu'il est indispensable pour exploiter les mesures du capteur ?

5 Piloter le SHTC3 en I²C, sans bibliothèque

Vous allez maintenant acquérir la température du SHTC3 **sans bibliothèque dédiée** : aucun pilote existant n'est autorisé. Vous écrivez vous-mêmes la communication avec le capteur, directement sur le bus I²C, à partir de la datasheet.

Configuration attendue : mesure en **mode normal**, une mesure **par seconde**, et capteur en **sleep** entre deux mesures. Chaque mesure est ensuite **envoyée au PC** par le port série, puis tracée en temps réel dans votre application de la partie 3.

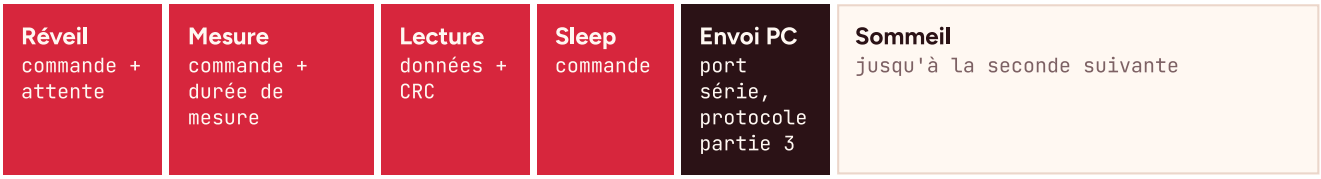


Figure 9 — Un cycle de mesure de 1 s (pas à l'échelle : la partie active du capteur dure environ 13 ms). La mesure est transmise au PC une fois le capteur rendormi. Les commandes et les délais sont à trouver dans la datasheet.

Ce qu'il faut chercher dans la datasheet

Les encadrés ci-dessous indiquent où se trouvent les informations utiles. Les adresses et les codes de commande ne sont volontairement pas recopiés : relevez-les vous-mêmes.

DATASHEET · INTERFACE I²C — § 4, § 5.1 · tableaux 6 et 8

- Le capteur accepte les modes Standard, Fast et Fast Mode Plus (horloge SCL jusqu'à 1 MHz), avec *clock stretching* optionnel.
- Son **adresse I²C sur 7 bits** est donnée au tableau 8.
- Le SHTC3 n'a pas de registres au sens classique : chaque action est une **commande de 16 bits**, envoyée poids fort en premier après l'en-tête d'écriture.

DATASHEET · MISE SOUS TENSION, SLEEP ET RÉVEIL § 5.2 · tableaux 5, 9 et 10

- À la mise sous tension, le capteur passe en état *idle* après t_{PU}. Il doit ensuite être placé en **sleep** par une commande dédiée (tableau 9).
- En sleep, le capteur ne répond plus : une **commande de réveil** (tableau 10) est obligatoire avant toute autre communication.
- Après le réveil, respectez le délai indiqué au tableau 5 avant d'envoyer la commande suivante.

DATASHEET · DÉCLENCHER UNE MESURE § 5.3, § 5.4 · tableaux 5 et 11, figure 7

- Un cycle de mesure enchaîne **quatre commandes** : réveil, mesure, lecture, sleep (figure 7).
- Le tableau 11 donne les commandes de mesure selon trois choix : mode (normal ou basse consommation), *clock stretching* activé ou non, température ou humidité lue en premier.
- Chaque commande déclenche toujours une mesure de température **et** d'humidité. La durée de mesure en mode normal figure au tableau 5.

DATASHEET · LIRE LE RÉSULTAT § 5.5, § 5.6

- Sans *clock stretching*, le capteur ne répond pas (NACK) tant que la mesure est en cours : attendez la durée maximale ou interrogez-le jusqu'à obtenir un ACK.
- La lecture renvoie **2 octets de données suivis d'1 octet de CRC**, puis à nouveau 2 octets et 1 CRC pour la seconde grandeur.
- Le maître peut interrompre la lecture après n'importe quel octet s'il n'a pas besoin de la suite.

DATASHEET · CRC ET CONVERSION § 5.10, § 5.11 · tableau 16

- Le CRC-8 couvre les deux octets de données qui le précèdent ; ses paramètres et deux exemples de contrôle sont donnés au tableau 16.
- La température s'obtient à partir de la valeur brute S_T sur 16 bits par la formule du § 5.11.

Accéder au bus I²C avec Arduino

La bibliothèque `Wire`, fournie avec le BSP, gère le protocole I²C : conditions START/STOP, adressage, acquittements. L'exemple ci-dessous n'est spécifique à aucun composant. Il montre comment écrire une valeur de 16 bits et lire des octets.

I2C_GENERIQUE.INO C / ARDUINO

```c
#include <Wire.h>

const uint8_t ADRESSE = 0x00;  // adresse I2C sur 7 bits : à trouver

// Écrit une commande de 16 bits, octet de poids fort en premier
bool ecrire16(uint16_t valeur) {
  Wire.beginTransmission(ADRESSE);
  Wire.write(valeur >> 8);
  Wire.write(valeur & 0xFF);
  return Wire.endTransmission() == 0;  // 0 : le composant a acquitté (ACK)
}

// Lit n octets ; renvoie le nombre d'octets réellement reçus
uint8_t lire(uint8_t *buf, uint8_t n) {
  uint8_t recus = Wire.requestFrom(ADRESSE, n);
  for (uint8_t i = 0; i < recus; i++) buf[i] = Wire.read();
  return recus;
}

void setup() {
  Serial.begin(115200);
  Wire.begin();            // le RP2040 est maître du bus
  Wire.setClock(400000);   // Fast Mode, 400 kHz
}
```

Code 3 — Écriture et lecture génériques sur le bus I²C.

ASTUCE

Vérifiez systématiquement la valeur renvoyée par `endTransmission()` et le nombre d'octets reçus. Un NACK signale une mauvaise adresse, un capteur endormi ou une mesure en cours. C'est votre premier outil de diagnostic.

Q15. Relevez dans la datasheet l'adresse I²C du capteur et les commandes de réveil, de mise en sleep et de mesure en mode normal que vous allez utiliser. Justifiez le choix de la commande de mesure.

Q16. Décrivez la séquence complète d'un cycle de mesure : commandes, octets lus et délais à respecter entre chaque étape.

Q17. Implémentez l'acquisition à 1 Hz avec mise en sleep entre deux mesures. Vérifiez d'abord les valeurs dans le Moniteur série.

Envoyez ensuite chaque mesure au PC et **tracez-la dans votre application**. Votre protocole de la partie 3 est-il encore adapté à la résolution du SHTC3 (0,01 °C) ? Adaptez-le si besoin et joignez une capture d'écran de la courbe obtenue.

Q18. Convertissez la valeur brute S_T en température × 10, **uniquement avec des entiers**. Quel type choisir pour les calculs intermédiaires, et pourquoi ?

Q19. Vérifiez le CRC de chaque lecture et rejetez les mesures invalides. Validez votre fonction avec les exemples du tableau 16.